16

Lecture 16

Network Functions Virtualisation (NFV)

مجازی‌سازی کارکردهای شبکه (NFV)

2G — 3G — 4G — **5G** — 6G

COURSE PART
Part III – Software-driven architectures (Lectures 15–18)

ESTIMATED STUDY TIME
6–8 hours (3 h reading, 3–5 h exercises)

DIFFICULTY
Intermediate

PREREQUISITES
Lectures 11 and 15; operating systems (processes, virtual memory); basic probability

In this lecture

1 Learning objectives . . . 2
2 Prerequisite knowledge . . . 2
3 Introduction and motivation . . . 3
4 Fundamental concepts . . . 4
5 Detailed technical explanation . . . 5
6 Mathematical models . . . 6
7 Network architecture and diagrams . . . 7
8 Step-by-step procedures . . . 8
9 Worked examples . . . 8
10 Technology comparison . . . 9
11 Key takeaways . . . 10
12 Exercises . . . 10
13 Further reading . . . 12

Original lecture notes for the course *Advanced Communication Networks*. The lecture sequence follows the course syllabus (26 lectures, 2G to 6G). Equations use identical notation in the English and Persian editions. Companion edition: Persian (`lecture-16-fa.pdf`).

1 Learning objectives

After studying this lecture you should be able to:

- **LO1** **Explain** why operators moved network functions from dedicated appliances to software on shared infrastructure.
- **LO2** **Describe** the ETSI NFV architectural framework: VNFs, NFVI and the MANO functions with their reference points.
- **LO3** **Compare** virtual machines and containers, and **explain** cloud-native network functions.
- **LO4** **Trace** the VNF lifecycle, including scaling and healing.
- **LO5** **Compute** availability of virtualised stacks, scaling needs and per-packet processing budgets.
- **LO6** **Relate** NFV to SDN and to the virtualised EPC and the cloud-native 5G core.

2 Prerequisite knowledge

You should know what an operating system kernel, a process and virtual memory are, and how series and parallel availability are combined (Lecture 11). SDN (Lecture 15) is the natural companion of NFV.

Key terms. The vocabulary of this lecture with its Persian equivalents:

Term	Persian	Meaning in this lecture
NFV	مجازی‌سازی کارکردهای شبکه	Running network functions as software on general-purpose servers.
VNF / VNFC	کارکرد شبکهٔ مجازی / مؤلفه	A virtualised network function / one of its components.
NFVI	زیرساخت NFV	Compute, storage, network hardware and the virtualisation layer.
MANO	مدیریت و هماهنگ‌سازی	NFV orchestrator, VNF manager and virtualised infrastructure manager.
VNFD / NSD	توصیفگر VNF / خدمت شبکه	Templates describing a VNF / a network service.
Hypervisor	ابرناظر	Software that runs virtual machines on a host.
Container	کانتینر	Isolated process group sharing the host kernel.
CNF	کارکرد شبکهٔ ابرزاد	Cloud-native network function built from containers/microservices.
SR-IOV / DPDK	فناوری‌های شتاب I/O	Direct NIC access for VMs / user-space packet processing.
Service function chain	زنجیرهٔ کارکرد خدمت	Ordered sequence of functions a flow traverses.

3 Introduction and motivation

Until the 2010s every network function – MME, HSS, gateway, firewall, load balancer – was a dedicated box: specialised hardware, a proprietary operating system and the function software, sold together. Adding capacity meant buying and installing more boxes; introducing a service meant integrating yet another appliance; each type had its own spares, tools and lifecycle. In 2012 a group of operators proposed, in a white paper that launched the ETSI NFV Industry Specification Group, to run these functions as software on standard servers in data centres, managed by common orchestration [1].

The promised benefits are those of cloud computing: shared hardware and lower cost, capacity that follows demand (elastic scaling), faster introduction of services, and automation. The costs are equally real: packet processing on general-purpose CPUs is harder than on ASICs, and availability now depends on a stack of hardware, virtualisation and software layers operated by different teams. The virtualised EPC was the first large mobile application; the 5G core was designed from the start as cloud-native software (Lectures 18–19).

4 Fundamental concepts

4.1 The ETSI NFV framework

The framework (Figure 1) [1] has three domains:

- **VNFs**, each with an element manager (EM) for its application-level configuration.
- **NFVI**: physical compute, storage and network resources, abstracted by a virtualisation layer into virtual resources.
- **MANO** [2]: the *NFV orchestrator* (NFVO) manages network services and global resources; the *VNF manager* (VNFM) manages the lifecycle of VNF instances; the *virtualised infrastructure manager* (VIM, e.g. an OpenStack or Kubernetes control plane) manages the NFVI resources.

Named reference points (Os-Ma, Or-Vnfm, Vi-Vnfm, Or-Vi, Ve-Vnfm, Nf-Vi, Vn-Nf, Vl-Ha) define who talks to whom, so that components from different vendors can be combined.

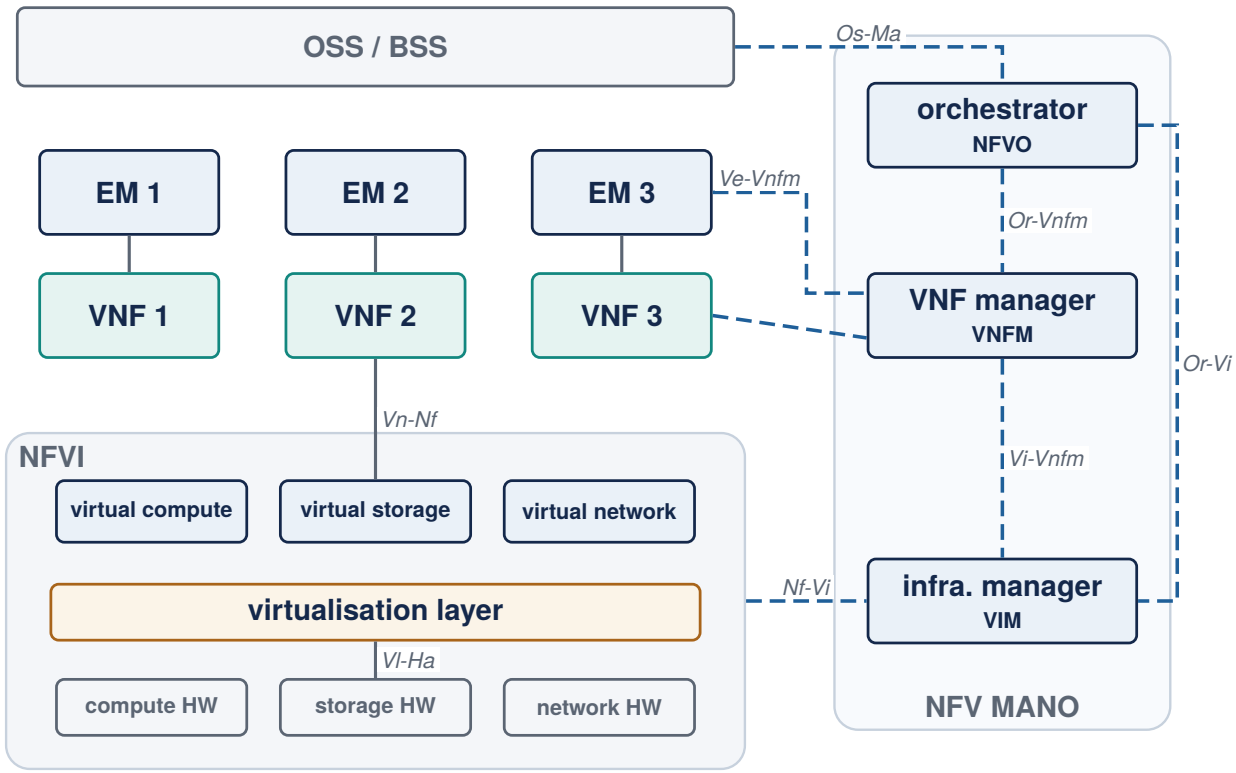


Figure 1. ETSI NFV architectural framework with its main reference points [1].

4.2 Descriptors and network services

A VNF is delivered as a package containing software images and a *VNF descriptor* (VNFD): its components (VNFCs), their resource needs, internal virtual links, connection points, scaling aspects and lifecycle scripts. A *network service* combines VNFs and the virtual links between them, described by a network service descriptor (NSD) and, where traffic must traverse functions in order, a forwarding graph. Descriptors make deployment declarative: the orchestrator turns a template into running instances.

4.3 Virtual machines and containers

A *virtual machine* runs a complete guest operating system on a hypervisor; it gives strong isolation and supports any OS, at the cost of memory and boot time. A *container* isolates a group of processes with kernel namespaces and control groups while sharing the host kernel; it starts in seconds and has little overhead (Figure 2). *Cloud-native network functions* (CNFs) are built as sets of containerised microservices, keep session state in separate data stores so that instances are disposable, and are deployed, scaled and healed by a container orchestrator such as Kubernetes.

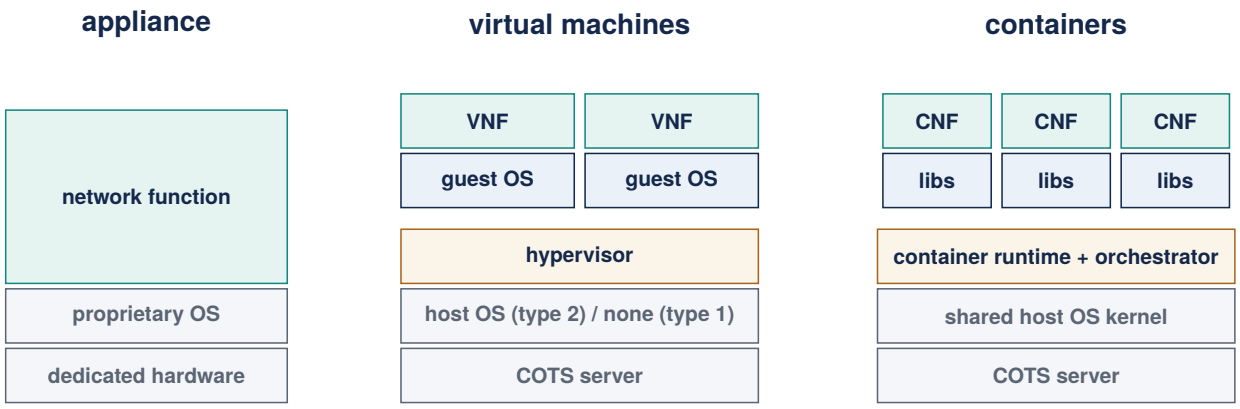


Figure 2. From appliances to virtual machines and containers.

5 Detailed technical explanation

5.1 Lifecycle management

The VNFM drives a VNF through the lifecycle operations of the MANO specifications [2]: *onboarding* (the package is validated and its images stored), *instantiation*, *scaling* out/in (adding or removing VNFC instances) or up/down (changing their resources), *healing* (replacing failed instances), *update/upgrade*, and *termination*. Scaling and healing can be triggered by the VNF or EM, by the VNFM from performance and fault data, or by the NFVO for a whole network service (Figure 4).

5.2 High-performance data planes

Gateways and UPFs must forward millions of packets per second. Plain virtual switching through the kernel and hypervisor cannot keep up, so NFVI nodes for user-plane functions use:

- **DPDK**: polling-mode drivers and packet processing in user space, avoiding interrupts and kernel copies;
- **SR-IOV**: the NIC presents virtual functions that are passed directly to VMs or containers;
- **CPU pinning and NUMA awareness**: dedicated cores on the socket attached to the NIC, huge pages for packet buffers;
- **Hardware offload**: smart NICs or programmable switches (Lecture 15) for encapsulation, classification or encryption.

These techniques trade some of NFV's flexibility (portability, overcommitment) for performance; the per-packet budget of Section 6.3 explains why they are needed.

5.3 Resilience in a virtualised world

In an appliance, redundancy was built into the box (redundant boards, synchronised state). In NFV, resilience is a property of the software design: instances are spread across hosts and sites (anti-affinity rules), state is replicated or externalised, and failed instances are healed by the orchestrator. Common-mode failures move up the stack: a faulty hypervisor update or orchestrator bug can affect every function at once.

5.4 NFV in mobile networks

The virtualised EPC (vEPC) packaged MME, S/P-GW, HSS and PCRF as VNFs, initially as "lift and shift" of appliance software into VMs. The 5G core takes the next step: network functions are defined as services (Lecture 18), implemented as CNFs and deployed on Kubernetes-based platforms, with the user plane at edge sites. In the RAN, virtualised and Cloud RAN run the baseband (CU and DU) on servers with acceleration hardware.

6 Mathematical models

6.1 Availability of a virtualised stack

Model. A VNF instance needs its host hardware, the virtualisation layer and its own software, with availabilities a_{hw}, a_{v} and a_{sw} (independent). In series, and with k independent redundant instances of which one suffices,

$$A_1 = a_{\mathrm{hw}}\, a_{\mathrm{v}}\, a_{\mathrm{sw}}, \qquad A_k = 1 - (1 - A_1)^k. \tag{1}$$

Example. $a_{\mathrm{hw}} = 0.999$, $a_{\mathrm{v}} = 0.9995$, $a_{\mathrm{sw}} = 0.999$: $A_1 = 0.9975$, i.e. 21.9 h of downtime per year – worse than each component. Two instances on independent hosts: $A_2 = 1 - (0.0025)^2 = 0.999994$, about 197 s per year, provided failover is instantaneous.

Interpretation. Redundancy at the instance level more than compensates for the less reliable layers, but only if failures are independent (different hosts, racks, sites) and state survives the failover.

6.2 Elastic scaling

Model. An instance serves μ requests/s; the operator keeps utilisation at most ρ_{t}. For offered load λ the number of instances is

$$n = \left\lceil \frac{\lambda}{\rho_{\mathrm{t}}\, \mu} \right\rceil. \tag{2}$$

If load can grow at rate g (requests/s per second) and a new instance takes T_{i} to become ready, the spare capacity must cover the growth during instantiation:

$$n\mu - \lambda \geq g\, T_{\mathrm{i}} \quad \Longleftrightarrow \quad \rho \leq 1 - \frac{g\, T_{\mathrm{i}}}{n\mu}. \tag{3}$$

Example. $\lambda = 40\,000\,\mathrm{s}^{-1}$, $\mu = 5000\,\mathrm{s}^{-1}$, $\rho_\mathrm{t} = 0.7$: $n = \lceil 11.4 \rceil = 12$. With $g = 100\,\mathrm{s}^{-2}$ and $T_\mathrm{i} = 60\,\mathrm{s}$ the headroom must be $6000\,\mathrm{s}^{-1}$, so $\rho \le 1 - 6000/60\,000 = 0.9$: the 70 % target is safe. Containers (T_i of seconds) allow much higher utilisation than VMs (minutes).

6.3 Per-packet processing budget

Model. At line rate R with packets of L bytes (plus 20 bytes of Ethernet preamble and inter-frame gap), one core at clock f has

$$c = \frac{f}{R/\left(8(L+20)\right)} \tag{4}$$

cycles per packet.

Example. $f = 2.5\,\mathrm{GHz}$, $R = 10\,\mathrm{Gbit/s}$, $L = 64$: 14.88 Mpps and $c = 168$ cycles – less than the cost of a single cache miss to main memory in the worst case. At 100 Gbit/s only 17 cycles remain (Figure 3).

Interpretation. Line-rate small-packet forwarding needs many cores in parallel, batching, cache-friendly data structures and offload; real traffic with larger average packets relaxes the budget considerably.

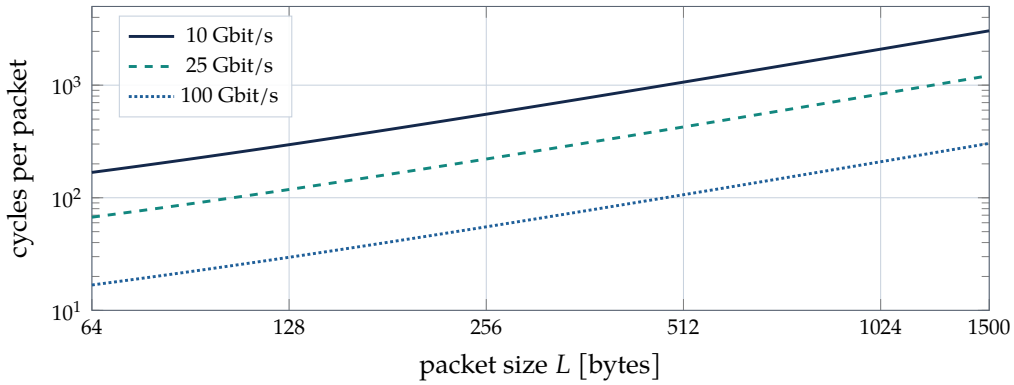


Figure 3. Cycles available per packet on one 2.5 GHz core at line rate (Eq. (4)).

6.4 Placement as bin packing

Model. VNFCs with CPU demands d_j must be placed on servers of capacity C. The number of servers is at least

$$m \ge \left\lceil \frac{\sum_j d_j}{C} \right\rceil, \tag{5}$$

and finding the minimum is the NP-hard bin-packing problem; heuristics such as first-fit decreasing come close. Anti-affinity (replicas on different servers) and latency constraints (user plane at the edge) add constraints that can increase m.

7 Network architecture and diagrams

An operator's NFV deployment typically consists of a few central data centres (control-plane functions, subscriber databases, OSS) and many edge sites (user-plane functions, RAN baseband), all

managed by a common orchestration layer. SDN controllers (Lecture 15) configure the data-centre fabrics and the transport between sites, so that the virtual links of network services are realised end to end. Figure 1 shows the functional view; physically, MANO components themselves run as virtualised applications on the same infrastructure.

8 Step-by-step procedures

Procedure 1 • Instantiating a network service

1. The operator (OSS) requests instantiation of a network service from the NFVO, referencing an onboarded NSD.
2. The NFVO checks resources, selects VIMs/sites and asks the VNFM of each VNF to instantiate it.
3. The VNFM requests a resource grant from the NFVO, then asks the VIM to create VMs or containers, networks and storage according to the VNFD.
4. The VIM allocates the resources on the NFVI and boots the images; the VNFM configures the instances and informs the EM.
5. The NFVO creates the virtual links between the VNFs (possibly through an SDN controller) and reports the service as instantiated.

Procedure 2 • Automatic scale-out

1. The VNF or EM reports load above a threshold defined in the VNFD (or the VNFM detects it from metrics).
2. The VNFM decides the scaling step and obtains a grant from the NFVO.
3. The VIM allocates resources and starts a new VNFC instance.
4. The VNFM configures it and adds it to the VNF's internal load distribution; traffic shifts to it.

9 Worked examples

Worked example 1 • Redundancy design

Problem. With the component availabilities of Section 6.1, compare one instance, two instances on the same host, and two instances on different hosts.
Solution. One: $A_1 = 0.9975$ (21.9 h/year). Same host: hardware and hypervisor are shared, so $A = a_{\text{hw}} a_{\text{v}} \left[1 - (1 - a_{\text{sw}})^2\right] = 0.9985$ (13.1 h/year): only software failures are covered. Different hosts: $A_2 = 0.999994$ (197 s/year). Anti-affinity is essential.

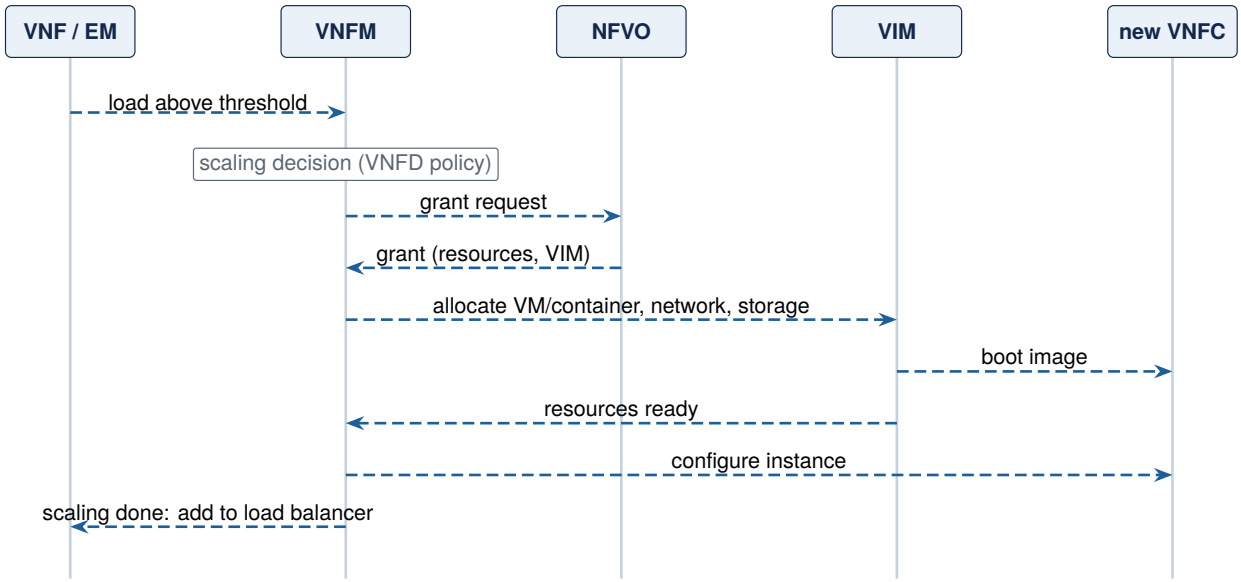


Figure 4. Simplified VNF scale-out (Procedure 2).

Worked example 2 • Scaling a signalling function

Problem. An AMF-like function must handle a busy hour of 40 000 procedures/s; each instance handles 5000/s. How many instances are needed at 70 % target utilisation, and what is the utilisation after one instance fails?
Solution. $n = 12$ (Eq. (2)); utilisation $40\,000/60\,000 = 67\,\%$. After a failure: $40\,000/55\,000 = 73\,\%$ – still below saturation, and the orchestrator heals the instance in the meantime.

Worked example 3 • Packing VNFCs

Problem. VNFCs need 6, 5, 5, 4, 4, 3, 3 and 2 cores; servers have 16 cores. Find the lower bound and a first-fit decreasing placement.
Solution. $\sum d_j = 32$, so $m \geq 2$. First-fit decreasing: server 1 gets 6, 5, 5 (16 cores); server 2 gets 4, 4, 3, 3, 2 (16 cores). Two servers, which is optimal – but with no spare capacity for healing.

10 Technology comparison

NFV and SDN. The two are complementary and independent: NFV changes *where* functions run (software on shared servers), SDN changes *how* forwarding is controlled (programmable devices, central control). Together they allow a network service – functions and the links between them – to be created by software.

11 Key takeaways

Table 1. Physical appliances, VM-based VNFs and cloud-native CNFs.

	Appliance	VNF (VMs)	CNF (containers)
Hardware	Dedicated, vendor-specific	Shared COTS servers	Shared COTS servers
Isolation	Physical	Hypervisor	Kernel namespaces, cgroups
Start-up time	Hours to days (install)	Minutes	Seconds
Scaling	Buy and install boxes	Add VMs (scale out/up)	Add pods automatically
State handling	Inside the box	Often inside the VM	Externalised data stores
Lifecycle	Vendor-specific tools	MANO (NFVO, VNFM, VIM)	Kubernetes + operators, CI/CD
Data-plane speed	ASIC line rate	Needs DPDK, SR-IOV	Needs DPDK, SR-IOV, offload

Key takeaways

- NFV runs network functions as software on shared infrastructure, managed by MANO (NFVO, VNFM, VIM).
- Descriptors (VNFD, NSD) make deployment declarative; lifecycle operations include scale and heal.
- Containers and cloud-native design (stateless instances, external state) underpin the 5G core.
- Series availability drops with each layer; independent redundant instances restore it: $A_k = 1 - (1 - A_1)^k$.
- Scaling: $n = \lceil \lambda / (\rho_t \mu) \rceil$ with headroom gT_i; packet budget $c = 8f(L + 20)/R$ cycles.

12 Exercises

Exercise 1 • Level 1 – conceptual • Who does what?

Which MANO component (NFVO, VNFM, VIM) is responsible for: (a) creating a virtual machine; (b) deciding to add a VNFC instance; (c) connecting several VNFs into a network service; (d) keeping an inventory of NFVI resources?

Exercise 2 • Level 1 – conceptual • VM or container?

Give one argument for running a virtualised HSS in VMs and one for running it as containers.

Exercise 3 • Level 2 – numerical • Same-host redundancy

Using Eq. (1), compute the annual downtime of three instances placed on three different hosts that share one power and network domain with availability 0.9999. Which term dominates?

Exercise 4 • Level 2 – numerical • Packet budget

A UPF core runs at 3 GHz and needs 400 cycles per packet. What throughput can one core sustain with 64-byte and with 800-byte packets? How many cores for 100 Gbit/s at 800 bytes?

Exercise 5 • Level 3 – analytical • Scaling lag

Load ramps at $g = 200\,\mathrm{s}^{-2}$. Instances serve $\mu = 4000\,\mathrm{s}^{-1}$. Compare the maximum safe utilisation for $n = 10$ instances when a new instance takes 120 s (VM) or 10 s (container) to become ready. Discuss what else limits the speed of scaling.

Exercise 6 • Level 3 – analytical • Placement with anti-affinity

The VNFCs of Example 3 consist of two replicas of a 6-core component (A), two of a 5-core component (B) and the rest (4, 4, 3, 3, 2) without constraints; replicas of the same component must be on different servers, and each server must keep 2 cores free for healing. Find a placement with the fewest 16-core servers.

Solutions

Solution to Exercise 1

(a) VIM; (b) VNFM (possibly at the request of the EM/VNF or NFVO); (c) NFVO; (d) VIM (with a global view kept by the NFVO).

Solution to Exercise 2

VMs: strong isolation for a security-critical database holding subscriber keys, and support for an existing vendor image. Containers: fast scaling and healing, smaller footprint and integration with cloud-native tooling, if the HSS software is designed with external, replicated state.

Solution to Exercise 3

Three independent instances: $1 - (0.0025)^3 \approx 1 - 1.6 \times 10^{-8}$. Shared domain in series: $A = 0.9999 \times (1 - 1.6 \times 10^{-8}) \approx 0.99990$, i.e. about 53 min/year, almost all due to the shared domain. Independence of failure domains matters more than the number of instances.

Solution to Exercise 4

One core: $3 \times 10^9 / 400 = 7.5$ Mpps. 64-byte packets: $7.5 \times 10^6 \times 84 \times 8 = 5.0$ Gbit/s; 800-byte packets: $7.5 \times 10^6 \times 820 \times 8 = 49$ Gbit/s (on the wire). For 100 Gbit/s at 800 bytes: $100/49 = 2.03$, i.e. 3 cores (more with headroom).

Solution to Exercise 5

Capacity $n\mu = 40\,000\,\mathrm{s}^{-1}$. VM: headroom $200 \times 120 = 24\,000$, so $\rho \leq 0.4$; container: $200 \times 10 = 2000$, so $\rho \leq 0.95$. Other limits: detection delay of the monitoring loop, image download, warm-up (caches, state synchronisation), load-balancer reconfiguration and the availability of free hardware at the site.

Solution to Exercise

Usable capacity is 14 cores per server; total demand 32, so at least 3 servers. Placement: server 1: A (6) + B (5) + 3 = 14; server 2: A (6) + B (5) + 3 = 14; server 3: 4 + 4 + 2 = 10. Anti-affinity holds (each server has one replica of A and B at most) and 2 cores remain free on each: three servers.

13 Further reading

The NFV architectural framework is ETSI GS NFV 002 [1] and management and orchestration ETSI GS NFV-MAN 001 [2]. Kreutz et al. [4] discuss the relationship between SDN and NFV; Rommer et al. [5] describe the cloud-native 5G core; Kleinrock [3] covers the queueing models used for dimensioning.

References

[1] ETSI, "Network functions virtualisation (NFV); architectural framework", European Telecommunications Standards Institute, Group Specification GS NFV 002, 2014.

[2] ETSI, "Network functions virtualisation (NFV); management and orchestration", European Telecommunications Standards Institute, Group Specification GS NFV-MAN 001, 2014.

[3] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley, 1975.

[4] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey", *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[5] S. Rommer, P. Hedman, M. Olsson, L. Frid, S. Sultana, and C. Mulligan, *5G Core Networks: Powering Digitalization*. Academic Press, 2020.